

F2Z

1

1

July 20, 2026

Abstract

Contents

1	Introduction	2
1.1	Acknowledgements	2
2	Preliminaries	2
3	$\mathbb{F}_2\mathbb{Z}$	2
3.1	Constrained codes with dual fields Albert: not sure dual is a good word: the two fields are just “different”, while dual things tend to have a close relation between each other	2
3.2	Tensor decomposition, direct product of relations, and interleaved codes . .	3
3.3	The main F2Z IOR	4
4	Virtualization of $\mathbb{F}_2$-linear combinations lifted to integers	5
5	Case-study: SHA-256 hashing	6
5.1	Arithmetization over $\mathbb{Z}[X]$	7
5.1.1	SHA-256 compression overview	7
5.1.2	Our arithmetization	8
5.2	End-to-end IOP	8
5.3	Experiments	8
A	Galois FRI-Binius	8

1 Introduction

1.1 Acknowledgements

2 Preliminaries

Given a prime q we let $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ be the ring homomorphism mapping an integer to its reduction modulo q . We further let π_0 denote the identity map on $\mathbb{Z}$, so that $\pi_0(x) = x$ for all $x \in \mathbb{Z}$. We let $\mathbb{F}_0 = \mathbb{Z}$.

Given a finite field $\mathbb{F}$ of characteristic q , let $a(X) \in \mathbb{Z}[X]$ be an irreducible polynomial over $\mathbb{F}_q$ **Albert: yes?**, so that $\mathbb{F} = \mathbb{Z}[X]/(q, a(X))$ **Albert: consider isomorphism, or assume all finite fields are represented like this.** Let $\psi_{\mathbb{F}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$ be the canonical homomorphism from $\mathbb{Z}[X]$ to $\mathbb{F}$ obtained by reducing modulo the ideal $(q, a(X))$, i.e. by reducing coefficients modulo q and then reducing the resulting polynomial modulo $a(X)$.

Interactive Oracle Proofs of Proximity (IOPP) Interleaved codes

Interactive Oracle Reductions (IOR)

3 $\mathbb{F}2\mathbb{Z}$

3.1 Constrained codes with dual fields **Albert: not sure dual is a good word: the two fields are just “different”, while dual things tend to have a close relation between each other**

Let $\ell \geq 0$ be a dimension parameter, and $m \geq 0$ a blocklength parameter. Let $\mathbb{F}$ be a field (not necessarily a prime field), and let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension k and blocklength m over $\mathbb{F}$. Let $k \geq 0$ be a public input length parameter, and let $Q \in \mathbb{F}[Y_1, \dots, Y_k, Z_1, \dots, Z_\ell]$ be a constraint polynomial **Ulrich: Of arbitrary degree, I suppose? Albert: Yes!** with coefficients in $\mathbb{F}$ and on $k + \ell$ variables. Consider the following relation:

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}}^Q := \left\{ \left(\begin{array}{l} \mathbf{x} = \mathbf{y} \in \mathbb{F}^k, \\ \mathbf{y} = \mathbf{f} \in \mathbb{F}^m, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \right) \mid \begin{array}{l} \text{Enc}_{\mathcal{C}}(\mathbf{f}) = \mathbf{f} \\ \wedge Q(\mathbf{y}, \mathbf{f}) = 0 \end{array} \right\}$$

When Q and $\mathbf{y}$ are such that $\mathbf{y} = (\mathbf{r}, \mu) \in \mathbb{F}^{\log \ell} \times \mathbb{F}$ and

$$Q(\mathbf{y}, \mathbf{f}) = \text{mle}[\mathbf{f}](\mathbf{r}) - \mu,$$

we denote $\mathcal{R}_{\mathcal{C}, \mathbb{F}}^Q$ by $\mathcal{R}_{\mathcal{C}, \mathbb{F}}^{\text{mle}}$.

Next let $\mathbb{F}_{2^*}$ be a finite field of characteristic 2, and let $\mathbb{F}_q$ be a finite field of prime order q or $\mathbb{Q}$. For intuition, $\mathbb{F}_{2^*}$ is the field over which we will commit our witness, and $\mathbb{F}_q$ is the field over which we prove linear relations on the witness. We let

$$\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2 \subseteq \mathbb{F}_{2^*}, \quad \pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q,$$

be reduction mod 2 and q maps.

Let $\mathcal{C} \subseteq \mathbb{F}_{2^*}^n$ be a linear code of dimension k and blocklength n over $\mathbb{F}_{2^*}$. Define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^*}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \mid \begin{array}{l} \text{Enc}_{\mathcal{C}}(\pi_2(\mathbf{f})) = f \\ \wedge \langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}$$

3.2 Tensor decomposition, direct product of relations, and interleaved codes

We fix a tensor decomposition of vectors $\mathbf{v} \in \mathbb{F}_{2^*}^\ell$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in \mathbb{F}_q^{\ell_1}, \mathbf{v}^{(2)} \in \mathbb{F}_q^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We assume $\mathcal{C} = \mathcal{C}_{\ell_1}^{\equiv \ell_2}$ is the ℓ_2 -dimensional interleaving of a code $\mathcal{C}_{\ell_1} \subseteq \mathbb{F}_{2^*}^{m/2^{\ell_2}}$ of dimension ℓ_1 and blocklength $m/2^{\ell_2}$. We then index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi_q(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$.

Define the following bivariate polynomial:

$$P(Y, Z) = (Y - 1) \cdot Z + 1$$

Note $P(Y, 0) = 1$ and $P(Y, 1) = Y$. We assume prover and verifier have access to an IOR $\prod_{\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}}^Q \rightarrow \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}}^{\text{eval}}}$ that reduces $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}}^Q$ to $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}}^{\text{eval}}$, where

$$Q(Y_1, \dots, Y_{\ell_1+1}, Z_1, \dots, Z_{\ell_1}) = \prod_{i=1}^{\ell_1} P(Y_i, Z_i) - Y_{\ell_1+1}.$$

We assume the prover and verifier have access to an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}}^{\text{eval}}$.

Further, we assume that prover and verifier have access to an IOR that reduces $\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^Q$ to $\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^{\text{mle}}$, such that, given an input

$$\left(\mathbf{y}^{(j)} \in \mathbb{F}_{2^*}^{\ell_1}, g^{(j)} \in \mathbb{F}_{2^*}^{m/\ell_2}, \mathbf{g}^{(j)} \in \mathbb{F}_{2^*}^{\ell_1} \right)_{j \in [\ell_2]} \in \left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^Q \right)^{\ell_2}$$

outputs

$$\left((\mathbf{r}, \mu_j) \in \mathbb{F}_{2^*}^{\log \ell_1}, g^{(j)} \in \mathbb{F}_{2^*}^{m/\ell_2}, \mathbf{g}^{(j)} \in \mathbb{F}_{2^*}^{\ell_1} \right)_{j \in [\ell_2]} \in \left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^{\text{mle}} \right)^{\ell_2}.$$

In words, the IOR outputs ℓ_2 MLE evaluation claims for the input witnesses $\mathbf{g}^{(j)}$ behind the input oracles $g^{(j)}$, all on the same evaluation point μ . [Albert: We could be more abstract with this IOR, but I find it difficult to tie it with the protocol in the next section then. Maybe better to write a remark](#) Looking ahead, we will compose this IOR ℓ_2 times in parallel, obtaining an IOR from $\left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^Q \right)^{\ell_2}$ to $\left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^{\text{mle}} \right)^{\ell_2}$, outputting the same evaluation point for the ℓ_2 output claims [Albert: preliminaries? Cite](#).

We describe one such IOR in ??.

3.3 The main F2Z IOR

We provide an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}}$ next.

Construction 3.1 (IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}}$). The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^*}^n$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}}$. Let $g \in \mathbb{F}_{2^*}$ be a nonzero element from $\mathbb{F}_{2^*}$ with sufficiently large multiplicative order, to be determined later.

1. The prover sends $\mu'_1, \dots, \mu'_{\ell_2} \in \mathbb{Z}$ such that $\langle \mathbf{f}_{*j}, \pi_q^{-1}(\mathbf{v}^{(1)}) \rangle = \mu'_j$.
2. The verifier checks that $\sum_{j \in [\ell_1]} \pi_q(\mu'_j) \cdot \mathbf{v}_j^{(2)} = \mu$. [Albert: We could have the prover commit to the \$\mu'_j\$ and have this check proved by the PCS \(a PCS that can prove linear constraints like WHIR\). This pays off for very large \$\ell\$.](#)
3. Prover and verifier define $\gamma_i = \pi_q^{-1}(\mathbf{v}_i^{(1)})$ for $i \in [\ell_1]$, and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu'_j} \quad \text{for all } j \in [\ell_2].$$

Note that this claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \mathbf{f}_{ij} + 1) = g^{\mu'_j} \in \mathbb{F}_{2^*}, \quad \text{for all } j \in [\ell_2].$$

To prove the above claim, prover and verifier engage in the IOR that reduces

$$\left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^Q\right)^{\ell_2} \quad \text{to} \quad \left(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^{\text{mle}}\right)^{\ell_2},$$

with inputs **Albert:** The reason I'm leaving this IOR abstract here is that I think there may be better ways to prove the above claim than a grand product GKR, so I want to write it modular

$$\left(\mathbf{x}' = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu'_j}), \mathbf{y}'_j = \text{Enc}_{\mathcal{C}_{\ell_1}}(\pi_2(\mathbf{f}_{*j})), \mathbf{w}'_j = \pi_2(\mathbf{f}_{*j})\right) \quad \text{for } j \in [\ell_2].$$

4. At the end of this IOR, they are left with the following claims to prove:

$$\left((\mathbf{r}', \mu'_j), \text{Enc}_{\mathcal{C}_{\ell_1}}(\pi_2(\mathbf{f}_{*j})), \pi_2(\mathbf{f}_{*j})\right) \in \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^*}}^{\text{eval}}, \quad \text{for } j \in [\ell_2]$$

for some evaluation points and evaluation values $(\mathbf{r}', \mu'_j)$ output during the IOR. Accordingly, the prover and the verifier engage in a IOPP to certify the validity of this last claim.

We now describe a GKR-based IOR reducing $(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^Q)^{\ell_2}$ to $(\mathcal{R}_{\mathcal{C}_{\ell_1}, \mathbb{F}_{2^*}}^{\text{eval}})^{\ell_2}$ of the required form.

Construction 3.2 (Grand product GKR). **Albert:** Todo: it's a standard grand product GKR so far – though there's lots of implementation-level optimizations it can undergo

Ulrich: Maybe to just write verbally, that we will use GKR for the grand product. And defer the explicit protocol description to the appendix or any later section? At this point it is maybe more important to clarify how useful the construction is because of packed commitment scheme over $\mathbb{F}_2$. **Albert:** Sounds good

4 Virtualization of $\mathbb{F}_2$ -linear combinations lifted to integers

In this section we expand relation $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}}$ to impose, instead of constraints of the form $\text{Enc}_{\mathcal{C}}(\pi_2(\mathbf{f})) = f$ and $\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu$, constraints as $\text{Enc}_{\mathcal{C}}(\pi_2(\mathbf{f})) = f$ and $\langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu$ for a matrix $M \in \mathbb{F}_2^{\ell' \times \ell}$. Precisely, given a new parameter ℓ' ,

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin-virtual}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}, \\ \mathbf{y} = f \in \mathbb{F}_{2^*}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(\pi_2(\mathbf{f})) = f \\ \wedge \langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu \end{array} \right\}$$

The relation has the tag “lin(ear)-virtual” referencing the intuition that $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin-virtual}}$ expresses a linear constraint on the vector $\mathbf{g} = \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \in \mathbb{F}_q^{\ell'}$, which is not committed

directly: only $\pi_2(\mathbf{f})$ is. Note that $\mathbf{g}$ is a linear expression over $\mathbb{F}_{2^*}$ of the elements in $\pi_2(\mathbf{f})$, lifted to the integers, and then projected to $\mathbb{F}_q$.

Albert: Would be good to add more detail in this paragraph The relation $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin-virtual}}$ allows to, for example, express linear constraints over $\mathbb{F}_q$ of the XOR of witness elements, while only committing (over $\mathbb{F}_{2^*}$) to the original elements. The same applies with other bit operations such as AND and Majority, by noting first that, given integers $x, y, z \in \{0, 1\}$:
Albert: Should reference some identity in preliminaries, or explain more here

$$\begin{aligned}\pi_q(x \text{ AND } y) &= 2^{-1} \cdot \pi_q(x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y))), \\ \pi_q(\text{Maj}(x, y, z)) &= 2^{-1} \cdot \pi_q(x + y + z - \pi_2^{-1}(\pi_2(x) + \pi_2(y) + \pi_2(z))),\end{aligned}$$

where 2^{-1} denotes the inverse of 2 modulo q . Hence AND and Majority can be expressed over $\mathbb{F}_q$ as linear expressions of a vectors of the form $\pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})))$, with $\mathbf{f}$ containing only the original elements for which we compute AND and Majority.

We next provide an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin-virtual}}$. Similarly as before, we assume our input vectors $\mathbf{v} \in \mathbb{F}_q^{\ell'}$ admit a tensor structure of dimensions ℓ_1, ℓ_2 with $\ell' = \ell_1 \cdot \ell_2$.

Construction 4.1 (IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin}}$). The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}$ and $\mathbf{y} = f \in \mathbb{F}_{2^*}^n$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^*}, \mathbb{F}_q}^{\text{lin-virtual}}$.

Define the vector $\mathbf{g} = (\mathbf{g}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^{\ell_1 \times \ell_2}$ as $\mathbf{g} = \pi_2^{-1}(M \cdot \pi_2(f))$. Prover and verifier execute **Construction 3.1** until Step 4, given inputs

$$\mathbf{x}' = (\mu, \mathbf{v}), \quad \mathbf{y}' = \text{Enc}_{\mathcal{C}}(\pi_2(\mathbf{f})), \quad \mathbf{w}' = \mathbf{g}.$$

Once in Step 4, prover and verifier are left with claims **Albert: we should ask that the IOR verifier doesn't query the oracle, otherwise this could be incomplete**

$$((\mathbf{r}', \mu'_j), \mathbf{y}'_j, \mathbf{w}'_j) \in \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^*}}^{\text{eval}}, \quad \text{for } j \in [\ell_2],$$

where $\mathbf{y}'_j = \text{Enc}_{\mathcal{C}_{\ell_1}}(\pi_2(\mathbf{f}_{*j}))$ and $\mathbf{w}'_j = \mathbf{g}_{*j}$, for $j \in [\ell_2]$.

1.

5 Case-study: SHA-256 hashing

Albert: Maybe Blake3 better? (When comparing to Flock), because of the manu adds mod 2^{32}

5.1 Arithmetization over $\mathbb{Z}[X]$

5.1.1 SHA-256 compression overview

We begin with a brief overview of SHA-256 to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of SHA-256, a *word* is a ≤ 32 string of bits, which we represent as a binary integer polynomial, i.e. as an element of $\{0, 1\}^{<32}[X]$. The SHA-256 compression function receives as input a 512-bit string, represented as sixteen *message words* $(M_t)_{t \in [16]} \in \{0, 1\}^{<32}[X]^{16}$. The algorithm also uses 64 fixed *round constants* $(K_t)_{t \in [64]} \in \{0, 1\}^{<32}[X]^{64}$.

We let

$$\begin{aligned} \psi_2 : \mathbb{Z}[X] &\rightarrow \mathbb{Z} \\ a(X) &\mapsto a(2) \end{aligned} \tag{1}$$

be the ring homomorphism sending integer polynomials to their evaluation at $X = 2$. Notice that, given $a \in \{0, 1\}^{<32}[X]$, the integer $\psi_2(x) \in \mathbb{Z}$ is the integer whose bit-representation coincides with the coefficients of a .

SHA-256 is built from the following six auxiliary functions. For $x, y, z \in \{0, 1\}^{<32}[X]$, define:

$$\begin{aligned} \Sigma_0(x) &:= \text{ROTR}^2(x) \text{ XOR } \text{ROTR}^{13}(x) \text{ XOR } \text{ROTR}^{22}(x) && \in \{0, 1\}^{<32}[X] \\ \Sigma_1(x) &:= \text{ROTR}^6(x) \text{ XOR } \text{ROTR}^{11}(x) \text{ XOR } \text{ROTR}^{25}(x) && \in \{0, 1\}^{<32}[X] \\ \text{Ch}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (\neg x \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \text{Maj}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \sigma_0(x) &:= \text{ROTR}^7(x) \text{ XOR } \text{ROTR}^{18}(x) \text{ XOR } \text{SHR}^3(x) && \in \{0, 1\}^{<32}[X] \\ \sigma_1(x) &:= \text{ROTR}^{17}(x) \text{ XOR } \text{ROTR}^{19}(x) \text{ XOR } \text{SHR}^{10}(x) && \in \{0, 1\}^{<32}[X] \end{aligned} \tag{2}$$

Albert: todo: define the bit operations

Message schedule. At the beginning of the SHA-256 compression function, the sixteen input message words $(M_t)_{t \in [16]}$ are expanded into a so-called *message schedule*, which consists of 64 integers of 32 bits $(W_t)_{t \in [64]} \in \mathbb{Z}^{64}$ defined as follows:

$$W_t := \begin{cases} \psi_2(M_t) & t \in [16], \\ \psi_2(W_{t-16}) + \psi_2(\sigma_0(W_{t-15})) + \psi_2(W_{t-7}) + \psi_2(\sigma_1(W_{t-2})) \pmod{2^{32}} & t \in \{17, \dots, 64\}. \end{cases} \tag{3}$$

Compression loop. After the message schedule is computed, SHA-256 performs the so-called *compression loop*. Throughout it, the function maintains an 8-word internal state, conventionally labeled as

$$(a, b, c, d, e, f, g, h) \in \{0, 1\}^{<32}[X]^8.$$

The compression loop iterates 64 rounds. At each round $t \in [64]$, the state entering that round is denoted $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in \{0, 1\}^{<32}[X]^8$. The per-round update is given by, for $t \in [64]$,

$$\begin{aligned} T_{1,t} &:= \psi_2(h_t) + \psi_2(\Sigma_1(e_t)) + \psi_2(\text{Ch}(e_t, f_t, g_t)) + \psi_2(K_t) + W_t \pmod{2^{32}} && \in \mathbb{Z}, \\ T_{2,t} &:= \psi_2(\Sigma_0(a_t)) + \psi_2(\text{Maj}(a_t, b_t, c_t)) \pmod{2^{32}} && \in \mathbb{Z}, \\ a_{t+1} &:= \psi_2^{-1}(T_{1,t} + T_{2,t} \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\ e_{t+1} &:= \psi_2^{-1}(\psi_2(d_t) + \psi_2(T_{1,t}) \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\ (b_{t+1}, c_{t+1}, d_{t+1}, f_{t+1}, g_{t+1}, h_{t+1}) &:= (a_t, b_t, c_t, e_t, f_t, g_t) && \in \{0, 1\}^{<32}[X]^6. \end{aligned}$$

The full SHA-256 compression function adds the initial state to the terminal state word-wise modulo 2^{32} (the so-called *feed-forward step*); we omit this step for brevity [Albert: Let's not do that](#). For our purposes, the output of the compression function is the final state $(a_{65}, \dots, h_{65})$.

5.1.2 Our arithmetization

5.2 End-to-end IOP

5.3 Experiments

[Albert: GHASH or b127?](#)

[Albert: Comparison with Flock on SHA256: we commit to 8x less bits. This leads to more than 2x memory reduction. 80% of prover costs are in the grand product GKR. Hence setting rate=1/4 or smaller has little impact on performance. This leads to smaller proofs. Prover time overall seems to be about 1.5-2.5x worse for us. I also switched to a better config of Ligerito \(called k=4 in impl\), which reduces proof size by almost 2x, but increases open time, which for us adds a negligible overhead, while in Flock it adds more overhead](#)

A Galois FRI-Binius

[Ulrich: Just notes, not sure if it worth to keep it in final doc](#)

Packed commitment scheme for $\mathbb{F}_{2^m}$.

For simplicity we assume packing across polynomials. (The single poly case goes likewise.) Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m polys of 2^n coefficients

$$b_i(X) = \sum_{j=0}^{2^n-1} b_{i,j} \cdot X^j \in \mathbb{F}_2[X]^{<2^n}, \quad i = 1, \dots, m,$$

coefficient-wise into a single poly

$$b(X) = b_1(X) \cdot \beta_1 + \dots + b_m(X) \cdot \beta_m$$

$$= \sum_{j=0}^{2^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot X^j \in \mathbb{F}_{2^m}[X]^{<2^n}.$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset F$ fixed, and has no other fixed points. For any point $z \in \mathbb{F}$, we extract the value of $b_i(z)$ from the values of the packed poly $b(X)$ along the Galois orbit

$$z, z^2, z^{2^2}, \dots, z^{2^{m-1}}$$

as follows. The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & & & \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(b(\phi^{-k} z) \right) = b_1(z) \cdot \phi^k \beta_1 + \dots + b_m(z) \cdot \phi^k \beta_m,$$

for any k . To extract the $b_1(z)$ term, we look for weights $c_1, \dots, c_m \in F$ which annihilate the orbits of the other $\beta_2, \dots, \beta_m$, concretely

$$\sum_{k=0}^{m-1} c_k \cdot \phi^k \beta_1 = 1, \quad \sum_{k=0}^{m-1} c_k \cdot \phi^k \beta_j = 0 \quad j = 2, \dots, m.$$

This is exactly the first row vector of M^{-1} . In general,

$$\begin{pmatrix} b_1(z) \\ b_2(z) \\ b_3(z) \\ \vdots \\ b_m(z) \end{pmatrix} = M^{-1} \cdot \begin{pmatrix} v_0 \\ \phi v_1 \\ \phi^2 v_2 \\ \vdots \\ \phi^{m-1} v_{m-1} \end{pmatrix},$$

where $v_k = b(\phi^{-k} z)$. **Ulrich:** Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.